

News Article Recommender System

CSC4352

Final Project Report

Ghita Nafa & Lina Harcharras

Table Of Contents

1. Project Overview 3
1.1 Datasets Used 3
1.2 Technology Stack 3
2. System Architecture 3
3. Core Components Implementation 4
3.1 Data Preprocessing Pipeline 4
3.2 Transformer Integration 4
3.3 FAISS Vector Database 4
3.4 Recommendation Algorithm 4
4. Implementation Details 5
4.1 Data Schema 5
4.2 Project Structure 5
5. User Interface Features 5
6. Advanced Features 5
References 6
Appendix 7
Appendix A: Embedding Generation (models/embeddings.py) 7
Appendix B: FAISS Index Creation and Search (models/vector_db.py) 7
Appendix C: Recommendation Algorithm (models/recommender.py) 8
Appendix D: User Profile Vector (models/recommender.py) 9
Appendix E: Text Preprocessing (utils/preprocessing.py) 10

1. Project Overview

This project implements an intelligent news recommendation system that leverages modern natural language processing techniques and transformer-based models. The system understands the semantic meaning of news articles and recommends relevant content based on article similarities using pretrained DistilBERT embeddings and FAISS similarity search.

1.1 Datasets Used

The system supports the following datasets as specified in the project requirements:

- **AG News:** 120,000 categorized news articles across 4 categories (World, Sports, Business, Sci/Tech)
- **Sample Dataset:** 15 hand-crafted articles for testing and demonstration

1.2 Technology Stack

- **Libraries:** HuggingFace Transformers
- **Model:** DistilBERT embeddings (768 dimensions)
- **Similarity Search:** FAISS IndexFlatIP
- **Frontend:** Streamlit news portal

2. System Architecture

The system follows the five-layer architecture specified in the project requirements:

1. **User Interface Layer:** Streamlit-based frontend for browsing articles, searching, and viewing recommendations
2. **Recommendation Engine:** Core logic for query processing, similarity computation, and ranking
3. **Embedding Layer:** DistilBERT transformer for semantic text representation (768-dim vectors)
4. **Vector Database:** FAISS index for efficient similarity search
5. **Data Storage:** News article database with metadata (title, content, category, source, date)

3. Core Components Implementation

3.1 Data Preprocessing Pipeline

The preprocessing pipeline implements the required text cleaning operations:

- Remove special characters and HTML tags
- Normalize whitespace
- Handle Unicode characters
- Tokenization using WordPiece (BERT's tokenizer) with truncation at 512 tokens

3.2 Transformer Integration

DistilBERT was chosen over BERT for faster iteration as recommended in the project specification:

Feature	BERT	DistilBERT
Parameters	110M	66M
Layers	12	6
Speed	Slower	60% faster
Performance	Higher accuracy	~97% of BERT

The embedding generation process follows the three steps specified: (1) Input Preparation with [CLS] and [SEP] tokens, (2) Forward Pass through transformer layers, (3) Mean Pooling to extract a single 768-dimensional vector.

3.3 FAISS Vector Database

The system uses FAISS IndexFlatIP (Inner Product) for exact cosine similarity search, as recommended for datasets under 1 million vectors. Vectors are normalized before adding to the index to enable cosine similarity via inner product.

Mathematical Foundation: Cosine similarity measures the angle between two vectors: similarity(A, B) = (A · B) / (||A|| × ||B||). Values range from 0 (unrelated) to 1 (identical). For normalized vectors, this simplifies to the dot product.

3.4 Recommendation Algorithm

The content-based filtering algorithm implements the steps specified in the project:

- User reads/selects Article X
- Extract embedding from Article X
- Query FAISS index for top-K similar articles
- Filter out already-read articles
- Rank remaining articles by similarity score
- Return top-N recommendations

4. Implementation Details

4.1 Data Schema

Articles follow the schema specified in the project requirements:

- **article_id:** Unique identifier
- **title:** Article headline
- **content:** Full article text
- **category:** Article category (World, Sports, Business, Sci/Tech, etc.)
- **source:** Data source (AG_News, Sample_News)
- **published_date:** Publication date

4.2 Project Structure

- **app.py:** Streamlit web application
- **config.py:** Configuration settings
- **data/data_loader.py:** Dataset loading utilities
- **models/embeddings.py:** DistilBERT embedding generation
- **models/vector_db.py:** FAISS vector database wrapper
- **models/recommender.py:** Recommendation engine
- **utils/preprocessing.py:** Text preprocessing pipeline

5. User Interface Features

The Streamlit interface implements the UI features specified in the project:

- Article display cards with title, content preview, source, and category
- Search bar for semantic query input
- Recommendation panel showing similar articles
- Similarity scores displayed as percentages
- Category filtering capability

6. Advanced Features

The system tracks user reading history to build preference profiles as specified in the advanced features section. User profile vectors are computed by aggregating embeddings of read articles:

User Profile Vector = Σ(article_embeddings_read) / N

Recommendations are then weighted toward the user's preference vector, implementing implicit feedback through article clicks.

References

Devlin et al. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." https://aclanthology.org/N19-1423.pdf
Sanh et al. (2019). "DistilBERT, a distilled version of BERT." emc2-neurips19-paper-33.pdf
Vaswani et al. (2017). "Attention Is All You Need." Attention is All you Need
FAISS: Facebook AI Similarity Search https://github.com/facebookresearch/faiss
HuggingFace Transformers https://huggingface.co/docs/transformers/

Appendix

Appendix A: Embedding Generation (models/embeddings.py)

```
def generate_embedding(self, text: str, normalize: bool = True) ->
np.ndarray:
    """Generate embedding for a single text."""
    # Tokenize
    inputs = self.tokenizer(
        text,
        return_tensors='pt',
        truncation=True,
        max_length=512,
        padding=True
    )
    inputs = {k: v.to(self.device) for k, v in inputs.items()}

    # Forward pass
    with torch.no_grad():
        outputs = self.model(**inputs)

    # Pool embeddings (mean pooling)
    attention_mask = inputs['attention_mask']
    token_embeddings = outputs.last_hidden_state
    mask_expanded = attention_mask.unsqueeze(-
1).expand(token_embeddings.size()).float()
    sum_embeddings = torch.sum(token_embeddings * mask_expanded,
dim=1)
    sum_mask = torch.clamp(mask_expanded.sum(dim=1), min=1e-9)
    embedding = (sum_embeddings / sum_mask).cpu().numpy()[0]

    # Normalize for cosine similarity
    if normalize:
        norm = np.linalg.norm(embedding)
        if norm > 0:
            embedding = embedding / norm

    return embedding
```

Appendix B: FAISS Index Creation and Search (models/vector_db.py)

```
def __init__(self, dimension: int = 768, index_type: str =
'flat_ip'):
    """Initialize FAISS index."""
    self.dimension = dimension

    if index_type == 'flat_ip':
```

```
        self.index = faiss.IndexFlatIP(dimension)  # Inner product
for cosine similarity
    elif index_type == 'flat_l2':
        self.index = faiss.IndexFlatL2(dimension)

    self.id_map: Dict[int, str] = {}  # Map FAISS indices to
article IDs

def search(self, query_embedding: np.ndarray, k: int = 10,
           exclude_ids: List[str] = None) -> List[SearchResult]:
    """Search for k nearest neighbors."""
    query = query_embedding.reshape(1, -1).astype(np.float32)

    # Search with extra results for filtering
    search_k = k + len(exclude_ids) if exclude_ids else k
    scores, indices = self.index.search(query, min(search_k,
self.size))

    results = []
    for score, idx in zip(scores[0], indices[0]):
        if idx == -1:
            continue
        article_id = self.id_map.get(int(idx))
        if exclude_ids and article_id in exclude_ids:
            continue
        results.append(SearchResult(article_id=article_id,
score=float(score), index=int(idx)))
        if len(results) >= k:
            break
```

Appendix C: Recommendation Algorithm (models/recommender.py)

```
def get_recommendations(self, article_id: str, n_recommendations:
int = 10,
                        exclude_read: bool = True) ->
List[Recommendation]:
    """Get article recommendations based on a seed article."""
    # Step 1: Get seed article embedding
    query_embedding = self._embeddings_cache[article_id]

    # Step 2: Determine articles to exclude
    exclude_ids = [article_id]
    if exclude_read:
        exclude_ids.extend(list(self.user_history))

    # Step 3: Query FAISS for similar articles
    search_results = self.vector_db.search(
```

```
        query_embedding,
        k=n_recommendations * 3,
        exclude_ids=exclude_ids
    )

    # Step 4-6: Build and rank recommendations
    recommendations = []
    for result in search_results:
        article = self.articles_db.get(result.article_id)
        if not article:
            continue

        recommendation = Recommendation(
            article_id=result.article_id,
            title=article['title'],
            content=article['content'],
            category=article.get('category', 'Unknown'),
```

Appendix D: User Profile Vector (models/recommender.py)

```
def get_user_profile_embedding(self) -> Optional[np.ndarray]:
    """Generate user profile embedding based on reading
history."""
    if not self.user_history:
        return None

    # Aggregate embeddings of read articles
    embeddings = []
    for article_id in self.user_history:
        if article_id in self._embeddings_cache:
            embeddings.append(self._embeddings_cache[article_id])

    if not embeddings:
        return None

    # User Profile Vector = Σ(article_embeddings) / N
    profile = np.mean(embeddings, axis=0)

    # Normalize
    norm = np.linalg.norm(profile)
    if norm > 0:
        profile = profile / norm

    return profile
def get_user_profile_embedding(self) -> Optional[np.ndarray]:
    """Generate user profile embedding based on reading
history."""
    if not self.user_history:
```

```
        return None

    # Aggregate embeddings of read articles
    embeddings = []
```

Appendix E: Text Preprocessing (utils/preprocessing.py)

```
class TextPreprocessor:
    def preprocess(self, text: str) -> str:
        """Apply full preprocessing pipeline."""
        # Remove HTML tags
        text = re.sub(r'<[^>]+>', '', text)

        # Remove URLs
        text = re.sub(r'http[s]?://\S+', '', text)

        # Remove email addresses
        text = re.sub(r'\S+@\S+', '', text)

        # Normalize unicode
        text = unicodedata.normalize('NFKD', text)

        # Remove special characters (keep letters, numbers, basic
punctuation)
        text = re.sub(r'[^\w\s.,!?-]', '', text)

        # Normalize whitespace
        text = ' '.join(text.split())

        return text.strip()
class TextPreprocessor:
    def preprocess(self, text: str) -> str:
        """Apply full preprocessing pipeline."""
        # Remove HTML tags
        text = re.sub(r'<[^>]+>', '', text)

        # Remove URLs
        text = re.sub(r'http[s]?://\S+', '', text)
```